

Report OOP – Lab03

Họ và tên: Nguyễn Sỹ Anh Đức

MSSV: 20225703

Table of Contents

I. New written code	2
1. Working with method overloading	2
2. Passing parameter	2
3. Classifier Member and Instance Member	4
4. Print and Search Cart	4
5. Implement the Store class	7
6. String, StringBuilder and StringBuffer	9
II. Debug	11
III. UML Diagram	13
1. Use-Case Diagram	13
2. Class Diagram	14

I. New written code

1. Working with method overloading

Tại class **Cart.java**: nạp chồng phương thức **addDigitalVideoDisc**.

1.1 Overloading by differing types of parameter

```
23● public void addDigitalVideoDisc(DigitalVideoDisc [] dvdList) {  
24     if (dvdList.length > MAX_NUMBERS_ORDERED) {  
25         System.out.println("The cart is almost full!");  
26     } else {  
27         for (int i = 0; i < dvdList.length; i++) {  
28             itemsOrdered[qtyOrdered] = dvdList[i];  
29             System.out.println(dvdList[i].getTitle() + " has been added!");  
30             qtyOrdered +=1 ;  
31         }  
32     }  
33 }  
34 }
```

1.2 Overloading by differing the number of parameters

```
35● public void addDigitalVideoDisc(DigitalVideoDisc dvd1, DigitalVideoDisc dvd2) {  
36     DigitalVideoDisc [] dvdList = {dvd1, dvd2};  
37     addDigitalVideoDisc(dvdList);  
38 }  
39 }
```

2. Passing parameter

- Thêm setter cho title của lớp **DigitalVideoDisc**.

```
32● public void setTitle(String title)  
33 {  
34     this.title = title;  
35 }  
36 }
```

- Tạo lớp **DigitalVideoDiscWrapper** để wrap lên class **DigitalVideoDisc** từ đó có thể thay đổi được giá trị qua hàm swap đúng ở class **TestPassingParameter** như dưới đây

```
4 public class DigitalVideoDiscWrapper {
5     DigitalVideoDisc dvd;
6
7     DigitalVideoDiscWrapper(DigitalVideoDisc dvd)
8     {
9         super();
10        this.dvd = dvd;
11    }
12 }
```

```
4 public class TestPassingParameter {
5
6     public static void main(String[] args) {
7         DigitalVideoDisc jungleDVD = new DigitalVideoDisc("Jungle");
8         DigitalVideoDisc cinderellaDVD = new DigitalVideoDisc("Cinderella");
9         DigitalVideoDiscWrapper wjungleDVD = new DigitalVideoDiscWrapper(jungleDVD);
10        DigitalVideoDiscWrapper wcinderellaDVD = new DigitalVideoDiscWrapper(cinderellaDVD);
11        // Wrong swap() function
12        swap(jungleDVD, cinderellaDVD);
13        System.out.println("Wrong swap jungle dvd title: " + jungleDVD.getTitle());
14        System.out.println("Wrong swap cinderella dvd title: " + cinderellaDVD.getTitle());
15        // Correct swap() function
16        swap(wjungleDVD, wcinderellaDVD);
17        System.out.println("Correct swapped jungle dvd title: " + wjungleDVD.dvd.getTitle());
18        System.out.println("Correct swapped cinderella dvd title: " + wcinderellaDVD.dvd.getTitle());
19
20        changeTitle(jungleDVD, cinderellaDVD.getTitle());
21        System.out.println("Change jungle dvd title: " + jungleDVD.getTitle());
22    }
23    // Wrong
24    public static void swap(Object o1, Object o2)
25    {
26        Object tmp = o1;
27        o1 = o2;
28        o2 = tmp;
29    }
30    // Correct
31    public static void swap(DigitalVideoDiscWrapper o1, DigitalVideoDiscWrapper o2)
32    {
33        DigitalVideoDisc tmp = o1.dvd;
34        o1.dvd = o2.dvd;
35        o2.dvd = tmp;
36    }
37    public static void changeTitle(DigitalVideoDisc dvd, String title)
38    {
39        String oldTitle = dvd.getTitle();
40        dvd.setTitle(title);
41        dvd = new DigitalVideoDisc(oldTitle);
42    }
43
44 }
```

3. Classifier Member and Instance Member

- Tại class **DigitalVideoDisc** thêm instance như sau

```
10     private static int nbDigitalVideoDiscs = 0;
11     private int id;
12
```

```
42•   public DigitalVideoDisc(String title) {
43       super();
44       this.title = title;
45       this.id = ++nbDigitalVideoDiscs;
46   }
47•   public DigitalVideoDisc(String title, String category, float cost) {
48       super();
49       this.title = title;
50       this.category = category;
51       this.cost = cost;
52       this.id = ++nbDigitalVideoDiscs;
53   }
54•   public DigitalVideoDisc(String title, String category, String director, float cost) {
55       super();
56       this.title = title;
57       this.category = category;
58       this.director = director;
59       this.cost = cost;
60       this.id = ++nbDigitalVideoDiscs;
61   }
62•   public DigitalVideoDisc(String title, String category, String director, int length, float cost) {
63       super();
64       this.title = title;
65       this.category = category;
66       this.director = director;
67       this.length = length;
68       this.cost = cost;
69       this.id = ++nbDigitalVideoDiscs;
70   }
71
```

4. Print and Search Cart

4.1 Print Cart

- Viết lại hàm **toString()** để thay đổi kiểu trả về của string ở trong class **DigitalVideoDisc**.

```
72•   @Override
73   public String toString()
74   {
75       return this.id + ". DVD: " + this.title +
76           " - Category: " + this.category +
77           " - Director: " + this.title +
78           " - DVD length: " + this.length +
79           " - Cost: " + this.cost + "$";
80   }
81
```

- Tạo phương thức **print()** in danh sách các mặt hàng trong class **Cart**.

```
70● public void print()
71 {
72     System.out.println("*****CART*****");
73     System.out.println("Ordered Items:");
74     for (int i = 0; i < qtyOrdered; i++)
75     {
76         System.out.println(itemsOrdered[i]);
77     }
78     System.out.println("Total cost: " + totalCost());
79     System.out.println("*****");
80 }
81
```

4.2 Search Cart

- Tìm kiếm theo ID và tìm kiếm theo Title ở trong class **Cart**.

```
82● public void searchByID(int id)
83 {
84     boolean found = false;
85     for (int i = 0; i < qtyOrdered; i++)
86     {
87         if (itemsOrdered[i].getId() == id)
88         {
89             System.out.println("Found" + itemsOrdered[i]);
90             found = true;
91         }
92     }
93     if (found==false)
94     {
95         System.out.println("Sorry, no DVDs were found that match the ID provided!");
96     }
97 }
98
99● public void searchByTitle(String keyword)
100 {
101     boolean matchFound = false;
102     for (int i=0; i < qtyOrdered; i++)
103     {
104         if (itemsOrdered[i].isMatch(keyword))
105         {
106             System.out.println("Found" + itemsOrdered[i]);
107             matchFound = true;
108         }
109     }
110     if (matchFound == false)
111     {
112         System.out.println("Sorry, no DVDs were found with \"" + keyword + "\" in the title!");
113     }
114 }
115
```

- `boolean isMatch(String title)` trong `DigitalVideoDisc` để kiểm tra xem đĩa có khớp với tiêu đề đã cho hay không.

```
37 public boolean isMatch(String keyword)
38 {
39     return this.title.toLowerCase().contains(keyword.toLowerCase());
40 }
41
```

- Tạo class `CartTest` để kiểm tra các hàm trên

```
5 public class CartTest {
6     public static void main(String[] args) {
7
8         Cart cart = new Cart();
9
10        DigitalVideoDisc dvd1 = new DigitalVideoDisc("The Lion King",
11            "Animation", "Roger Allers", 87, 19.95f);
12        cart.addDigitalVideoDisc(dvd1);
13
14        DigitalVideoDisc dvd2 = new DigitalVideoDisc("Star War",
15            "Science Fiction", "George Lucas", 87, 24.95f);
16        cart.addDigitalVideoDisc(dvd2);
17
18        DigitalVideoDisc dvd3 = new DigitalVideoDisc("Aladin",
19            "Animation", 18.99f);
20        cart.addDigitalVideoDisc(dvd3);
21
22        cart.print();
23
24        cart.searchByID(3);
25        cart.searchByTitle("Lion");
26    }
27 }
```

5. Implement the Store class

- Tạo class **Store**, chứa thuộc tính **itemsInStore[]** là một mảng các DVD có sẵn trong cửa hàng.
- Thêm 2 phương thức **addDVD** và **removeDVD**

```
7 public class Store {
8     // List DVDs
9     private List<DigitalVideoDisc> itemsInStore = new ArrayList<DigitalVideoDisc>();
10
11     public void addDVD(DigitalVideoDisc dvd)
12     {
13         int index = itemsInStore.indexOf(dvd);
14         if (index != -1) {
15             System.out.println(dvd.getTitle() + " is already in the store.");
16         } else {
17             itemsInStore.add(dvd);
18             System.out.println(dvd.getTitle() + " has been added to the store.");
19         }
20     }
21
22     public void removeDVD(DigitalVideoDisc dvd)
23     {
24         boolean removed = itemsInStore.remove(dvd);
25         if(removed)
26         {
27             System.out.println(dvd.getTitle() + " has been removed from the store.");
28         } else {
29             System.out.println(dvd.getTitle() + " is not found in the store.");
30         }
31     }
32
33     public void print() {
34         for (int i=0; i < itemsInStore.size(); i++)
35         {
36             System.out.println((i+1) + ". " + itemsInStore.get(i));
37         }
38     }
39 }
```


- Tạo class **StoreTest** để kiểm tra 2 phương thức trên

```
5 public class StoreTest {
6     public static void main(String[] args) {
7         Store store = new Store();
8
9         // Create new dvd objects and add them to the store
10        DigitalVideoDisc dvd1 = new DigitalVideoDisc("The Lion King",
11            "Animation", "Roger Allers", 87, 19.95f);
12        store.addDVD(dvd1);
13
14        DigitalVideoDisc dvd2 = new DigitalVideoDisc("Star Wars",
15            "Science Fiction", "George Lucas", 87, 24.95f);
16        store.addDVD(dvd2);
17
18        DigitalVideoDisc dvd3 = new DigitalVideoDisc("Aladin",
19            "Animation", 18.99f);
20        store.addDVD(dvd3);
21
22        store.print();
23
24        store.addDVD(dvd3);
25        store.removeDVD(dvd3);
26
27        store.print();
28
29        store.addDVD(dvd3);
30
31        store.print();
32    }
33 }
```


6. String, StringBuilder and StringBuffer

- Tạo một lớp mới tên là **ConcatenationInLoops** để kiểm tra thời gian xử lý khi xây dựng chuỗi String bằng toán tử **+**, **StringBuilder** và **StringBuffer**.

```
5 public class ConcatenationInLoops {
6     public static void main(String[] args) {
7         Random r = new Random(123);
8
9         // Using + operator
10        long start = System.currentTimeMillis();
11        String s = "";
12        for (int i = 0; i < 65536; i++) {
13            s += r.nextInt(2);
14        }
15        System.out.println("Using + operator: " + (System.currentTimeMillis() - start) + "ms");
16
17        // Using StringBuffer
18        r = new Random(123);
19        start = System.currentTimeMillis();
20        StringBuffer sb = new StringBuffer();
21        for (int i = 0; i < 65536; i++) {
22            sb.append(r.nextInt(2));
23        }
24        s = sb.toString();
25        System.out.println("Using StringBuffer: " + (System.currentTimeMillis() - start) + "ms");
26
27        // Using StringBuilder
28        r = new Random(123);
29        start = System.currentTimeMillis();
30        StringBuilder sb2 = new StringBuilder();
31        for (int i = 0; i < 65536; i++) {
32            sb2.append(r.nextInt(2));
33        }
34        s = sb2.toString();
35        System.out.println("Using StringBuilder: " + (System.currentTimeMillis() - start) + "ms");
36    }
37 }
38
```

- Tạo một class mới là **GarbageCreator** đọc tệp văn bản vào 1 chuỗi dùng +

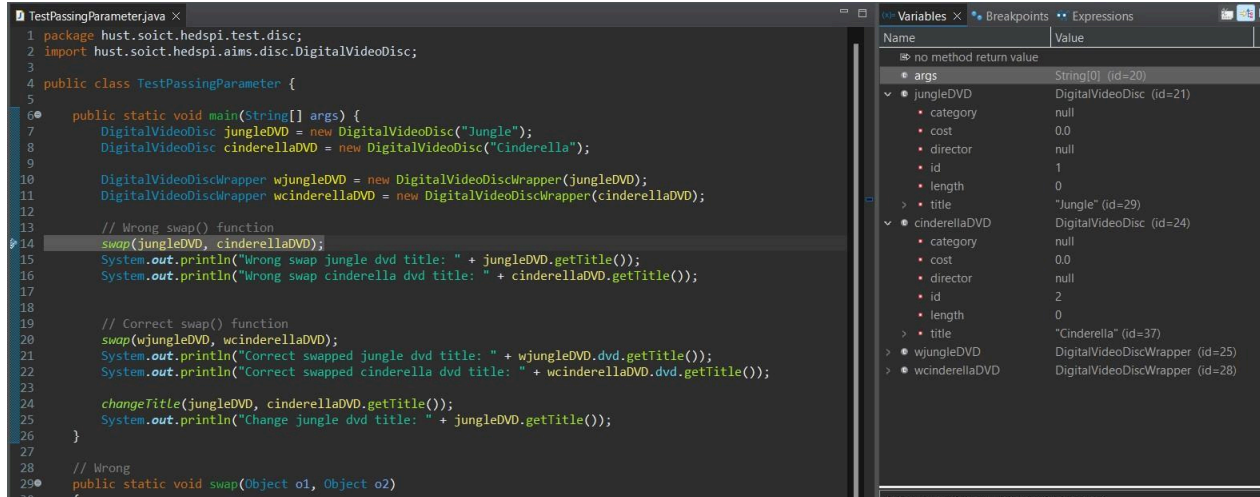
```
7 public class GarbageCreator {
8     public static void main(String[] args) {
9
10         String filename = "src/test.txt";
11         byte[] inputBytes = { 0 };
12         long startTime, endTime;
13
14         try {
15             inputBytes = Files.readAllBytes(Paths.get(filename));
16         } catch (IOException e) {
17             // TODO Auto-generated catch block
18             e.printStackTrace();
19         }
20
21         startTime = System.currentTimeMillis();
22         String outputString = "";
23         for (byte b : inputBytes) {
24             outputString += (char)b;
25         }
26         endTime = System.currentTimeMillis();
27         System.out.println(endTime - startTime);
28
29     }
30 }
31 }
```

- Tạo một class mới là **NoGarbage** đọc tệp văn bản vào 1 chuỗi dùng **StringBuilder**

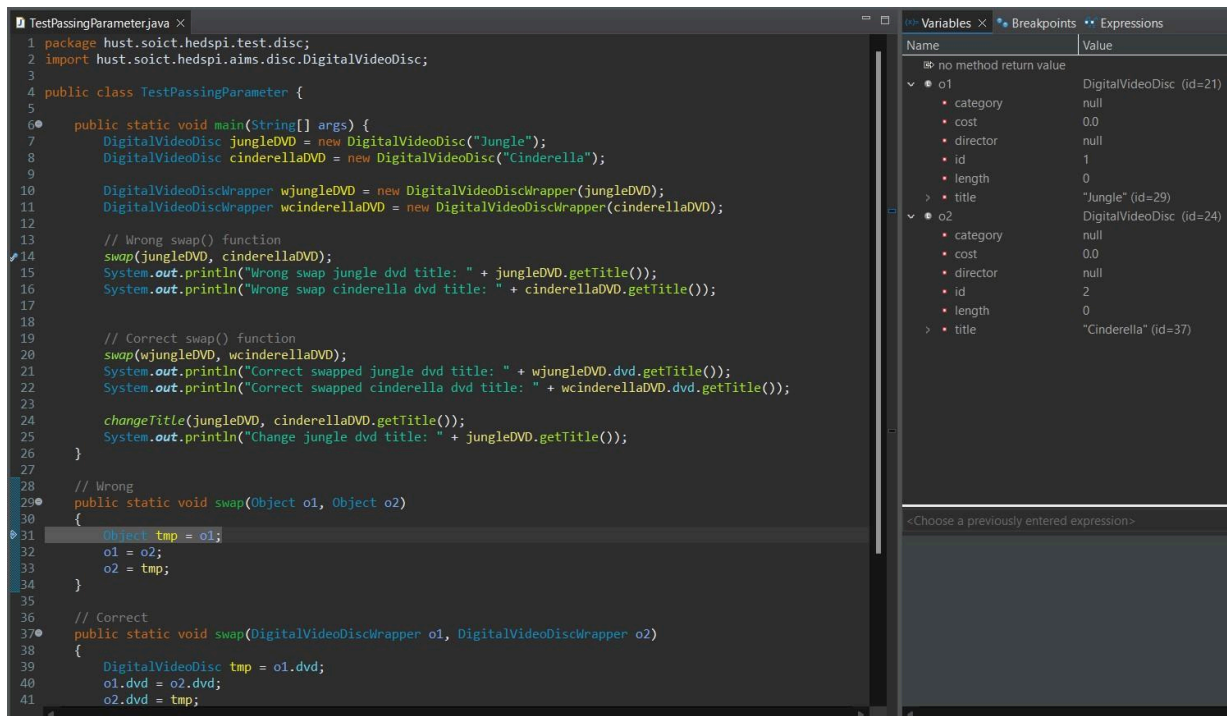
```
7 public class NoGarbage {
8     public static void main(String[] args) {
9
10         String filename = "src/test.txt";
11         byte[] inputBytes = { 0 };
12         long startTime, endTime;
13
14         try {
15             inputBytes = Files.readAllBytes(Paths.get(filename));
16         } catch (IOException e) {
17             // TODO Auto-generated catch block
18             e.printStackTrace();
19         }
20
21         startTime = System.currentTimeMillis();
22         StringBuilder outStringBuilder = new StringBuilder();
23         for (byte b : inputBytes) {
24             outStringBuilder.append((char)b);
25         }
26         endTime = System.currentTimeMillis();
27         System.out.println(endTime - startTime);
28
29     }
30 }
31 }
32 }
```

II. Debug

- Debug tại class **TestPassingParameter** với hàm **swap()**
- Đặt breakpoint tại **swap(jungleDVD, cinderellaDVD);**



- **Step into (F5)** để nhảy vào hàm



- Step over (F6) để chạy dòng lệnh tiếp theo

The screenshot shows an IDE with a Java file named `TestPassingParameter.java`. The code defines a `main` method and a `swap` method. The `main` method creates two `DigitalVideoDisc` objects, `jungleDVD` and `cinderellaDVD`, and wraps them in `DigitalVideoDiscWrapper` objects. It then calls `swap` twice: first with the wrapper objects, and then with the original `DigitalVideoDisc` objects. The `swap` method is designed to swap the `dvd` property of the wrapper objects. The `main` method also prints the titles of the discs before and after the swaps.

The Variables/Expressions view on the right shows the state of the program. It lists variables `o1` and `o2`, which are instances of `DigitalVideoDisc`. The `o1` object has properties: `category` (null), `cost` (0.0), `director` (null), `id` (1), `length` (0), and `title` ("Jungle" (id=29)). The `o2` object has properties: `category` (null), `cost` (0.0), `director` (null), `id` (2), `length` (0), and `title` ("Cinderella" (id=37)).

- Quan sát giá trị của các biến và biểu thức trong **Variables/Expression View**
- Thay đổi giá trị của biến

This close-up screenshot shows the Variables/Expressions view. It displays a table with two columns: **Name** and **Value**. The table shows the state of the program at a specific point. The variable `o1` is expanded, showing its properties: `category` (null), `cost` (0.0), `director` (null), `id` (1), `length` (0), and `title` ("abc" (id=41)).

- Kết quả

```
TestPassingParameter [Java Applicat
jungle dvd title: abc
jungle dvd title: cinderella
```

III. UML Diagram

1. Use-Case Diagram



2. Class Diagram

